Kassetten i ABC-datorerna

Signalen, blocken, filerna och verktygen

ABC80, ABC 800 C/M, ABC 802 och ABC 806

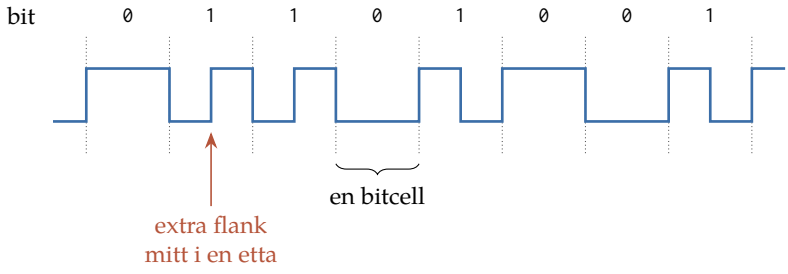
ABC80 och datorerna i ABC 800-serien sparar program och data på en vanlig ljudkassett. Här beskrivs hur: vilken signal som står på bandet, hur den blir block och filer, hur ROM:en skriver och läser den, och hur verktygen i `kassett/` flyttar filerna mellan bandet och dagens datorer. Uppgifterna kommer ur ROM:erna, ur schemat för ABC 800 och ur Markesjö 1978. Det som inte är belagt är märkt med (?).

	vad	*hur*	*skiljer sig*
1 Signal	bitcellerna på bandet	FM, flank i varje cells början	ABC80: programvaran; 800: SIO och TTL
2 Block	256 byte med inramning	synk, STX, data, ETX, summa	inledningen och slutet
3 Fil	filhuvud och datablock	namn och blocknummer	800: tidkonstanten i filhuvudet
4 Innehåll	text, BAC, data	teckenkod och radslut	ABC80-BASIC och BASIC II

Figur 1. En fil på bandet i fyra lager. Avsnitten nedan går uppifrån och ned.

1 Signalen

Signalen är FM (bifas): den vänds i början av varje bitcell, och en etta vänds en gång till mitt i cellen. Det är alltså antalet flanker per cell, inte nivån, som bär biten, och därför spelar det ingen roll om bandspelaren vänder signalen. Den minst signifikanta biten skrivs först.



Figur 2. Bitarna 0 1 1 0 1 0 0 1 i FM. De prickade linjerna är cellgränserna, och där vänds signalen alltid. En etta har en flank till i mitten.

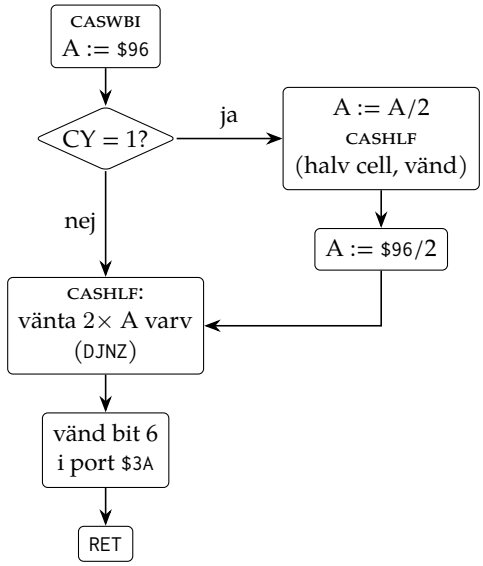
Bitcellens längd beror på maskinen. I ABC80 räknar programmet själv ut tiden mellan flankerna i T-cykler (klockcykler); i ABC 800-serien ger räknaren CTC dubbla bittakten, och tidkonstanten *tk* bestämmer den.

Maskin	Bitcell	Klocka	Tid	Bit/s
ABC80, nolla	3993 T	2,9952 MHz	1,333 ms	750
ABC80, etta	4065 T	2,9952 MHz	1,357 ms	737
ABC 800, tk 129	32×129 T	3 MHz	1,376 ms	727
ABC 800, tk 40	32×40 T	3 MHz	0,427 ms	2 344

Luxor anger 700 respektive 700 och 2 400 baud. I ABC80 är en etta lite längre än en nolla, eftersom ROM:en gör två anrop i stället för ett (i version 9913 är den 8 T kortare). I ABC 800-serien är alla celler lika långa.

2 Hur ABC80 skriver

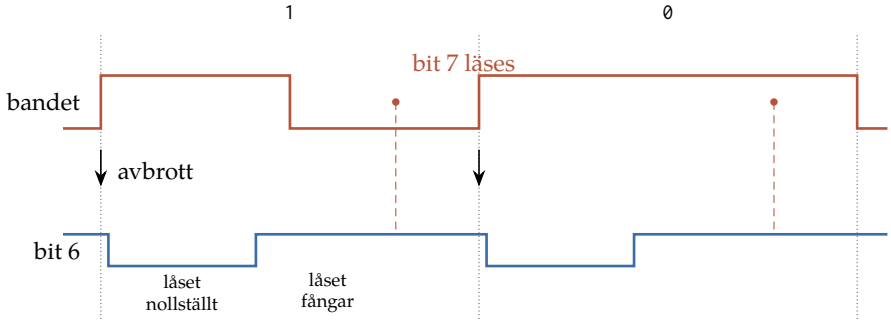
ABC80 har ingen krets för kassetten utöver tre bitar i PIO:ns port B, `$3A`: bit 5 drar motorreläet, bit 6 är skrivsignalen och bit 7 den lästa signalen. Allt annat gör ROM:en. Rutinen CASWBI skriver biten i CY, och CASHLF väntar och vänder signalen.



Figur 3. En bit från ABC80. En nolla är ett varv i CASHLF med `$96`: en hel cell och sedan en flank. En etta är två varv med hälften, alltså två flanker. CASWBY skriver en byte bit för bit med början i bit 0.

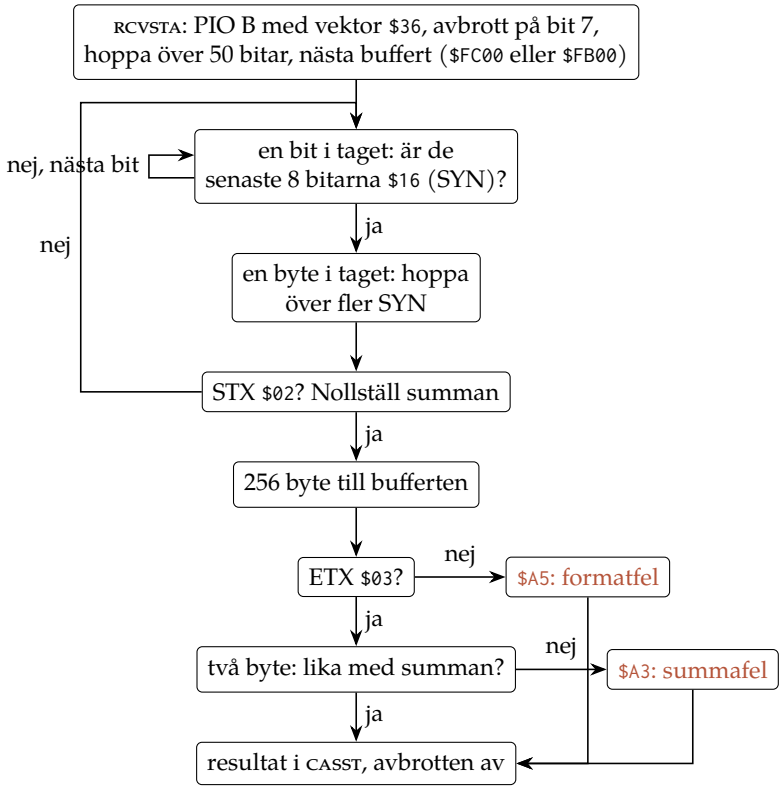
3 Hur ABC80 läser

Läsningen sker med avbrott. PIO B ger ett avbrott på flanken i början av varje cell, och rutinen CASINT tittar sedan efter om det har kommit en flank till. Bit 6, som skriver vid skrivning, styr vid läsning en låskrets som fångar flankerna (Markesjö 1978, s. 164–167).



Figur 4. CASINT, steg för steg. (1) Avbrottet kommer på flanken i cellens början. (2) Bit 6 sätts till 0, och låskretsen hålls nollställd `$78` varv i `DJNZ`, ungefär 0,5 ms. (3) Bit 6 sätts till 1, och låset kan fånga en flank. (4) Efter lika lång tid till, ungefär 3/4 cell från början, läses bit 7. Har en flank kommit i mitten av cellen är biten 1 (CY är den inverterade bit 7). Att låset fungerar just så är en tolkning (?).

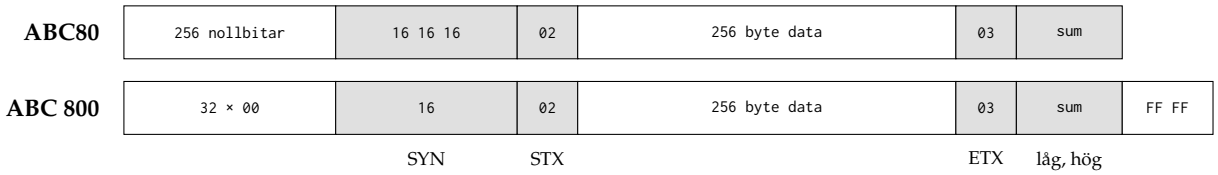
Bitarna samlas till block av en korutin: RCVBLK är skriven som ett vanligt program, men RCVBIT och RCVBYT sparar var den står och avslutar avbrottet. När nästa bit eller byte är klar hoppar CASINT tillbaka dit.



Figur 5. RCVBLK, korutinen som tar emot ett block. Varje pil nedåt i den vänstra spalten väntar på en eller åtta nya avbrott. Programmet (CASRBL) ger fel 42 om blocket inte är klart inom ungefär 8,5 s.

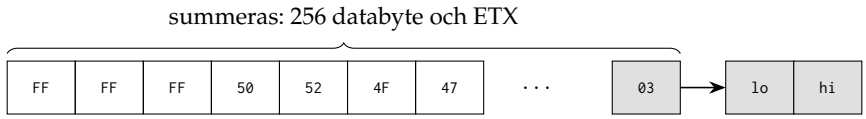
4 Blocket och summan

Ett block har 256 byte, inramade av en inledning med nollbitar, synk och STX före och ETX och en summa efter. De två maskinerna ramar in nästan lika.



Figur 6. Blocket i ABC80 (CASWBL) och i ABC 800-serien (CASWR). 256 nollbitar är detsamma som 32 nollbyte. Vid läsning letar SIO:n i ABC 800 efter `16 02`, så ett block från ABC80 duger, och verktygen godtar båda inledningarna.

Summan är 16 bitar, låg byte först, och räknas över de 256 databytena och ETX: summan är $3 + d_0 + d_1 + \cdots + d_{255}$, räknat modulo $65\,536$. I ABC80 syns det i CASINT, som lägger till varje mottagen byte, och i RCVBLK, som nollställer summan efter STX och läser av den efter ETX.

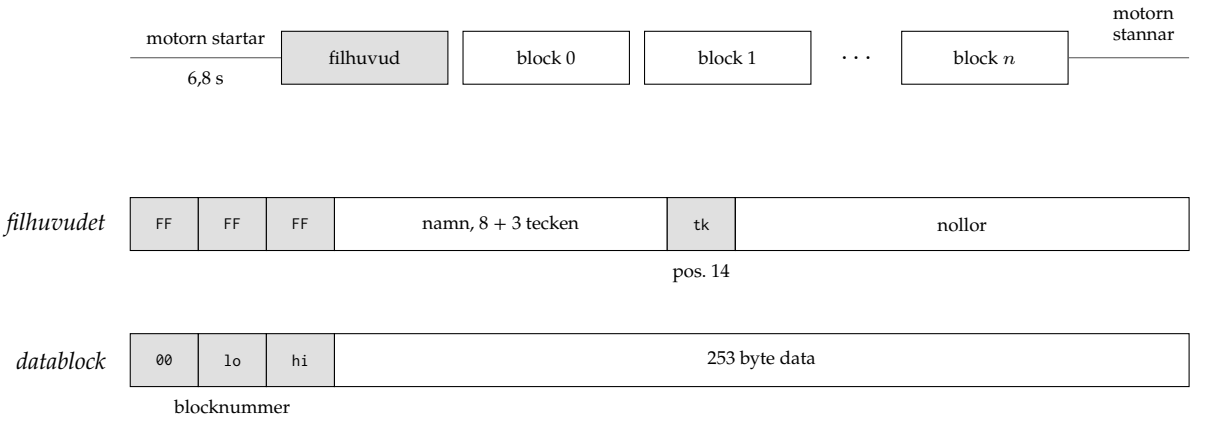


Figur 7. Summan. Ett block med `FF` och sedan 255 nollor ger $255 + 3 = 258 =$ `$0102`, alltså `02 01` på bandet.

Ett block från ABC80 är $256 + 8 \times 264 = 2\,368$ bitar och tar ungefär 3,2 s.

5 Filen

En fil är ett filhuvud (namnblocket) och sedan datablock, numrerade från 0. Före filhuvudet låter ROM:en motorn gå i ungefär 6,8 s så att bandet kommer upp i fart, och motorn går sedan hela tiden som filen är öppen. Skriver programmet sällan blir det tystnad mellan blocken.

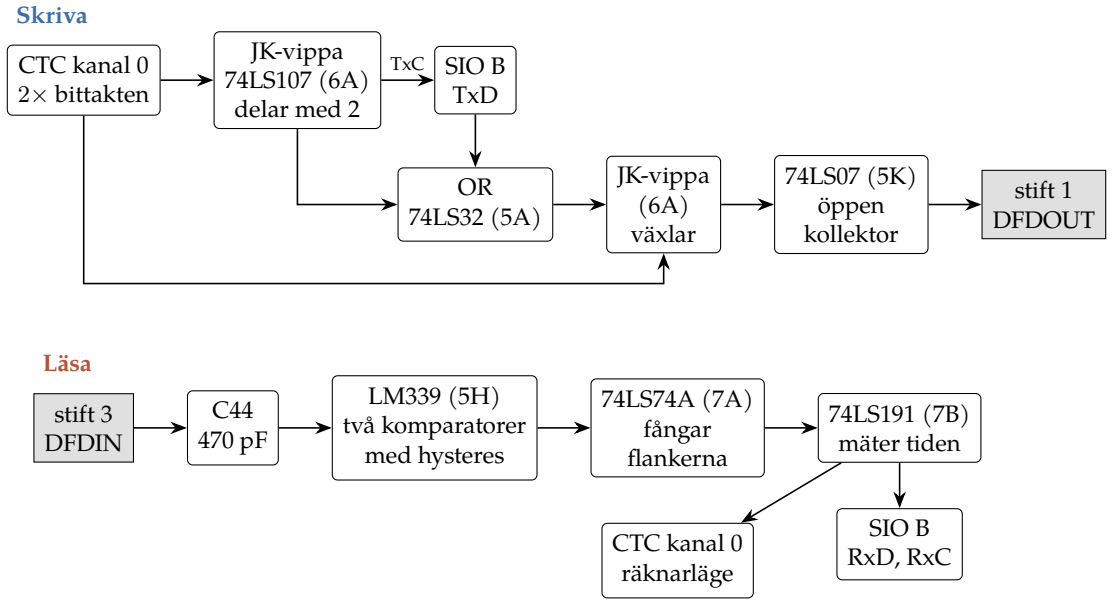


Figur 8. En fil på bandet, filhuvudet och ett datablock. Namnet fylls ut med blanksteg. Position 14 är tidkonstanten för datablocken i ABC 800-serien; ABC80 skriver en nolla där.

När ABC80 läser växlar ROM:en mellan två buffertar, `$FB00` och `$FC00`. Läser programmet långsamt går bandet vidare medan båda är fulla, och nästa block har då fel nummer (fel 37). Kommer inget block inom ungefär 8,5 s blir det fel 42.

6 ABC 800-serien

ABC 800 C/M, 802 och 806 skriver samma sorts FM, men med kretsar i stället för program. SIO:ns kanal B går i teckensynkront läge med synken `16 02`, och FM-kodningen görs med vanliga TTL-kretsar på processorkortet (schemat för ABC 800, blad 7). DTR i SIO:n drar motorreläet.



Figur 9. FM i ABC 800-serien. Vid skrivning delar en vippa CTC:ns klocka till bitklockan TxC. OR-grinden släpper igenom antingen den eller TxD till nästa vippa, som växlar: alltid vid cellgränsen och mitt i cellen när biten är 1. Vid läsning mäter en räknare tiden mellan flankerna och gör om dem till RxD och RxC. Hur räknaren laddas går inte att läsa i skanningen (?), och klockans väg till den andra vippan är förenklad.

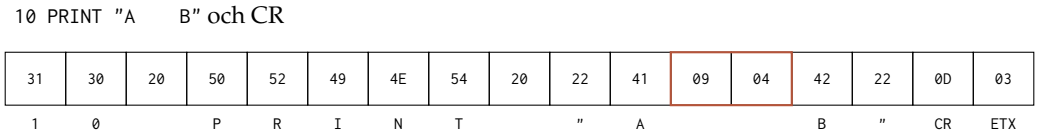
Bitcellen blir $2 \times 16 \times tk$ T vid 3 MHz. Filhuvudet skrivs och läses alltid med tk 129, ungefär 700 baud, och anger själv tidkonstanten för datablocken på position 14. ABC 802 skriver 40, ungefär 2 400 baud. Ett band från ABC80 har en nolla där, och då gör CASCTC ingenting: hela filen läses i 700 baud. Därför läser ABC 800 band från ABC80.

Maskin	Var koden ligger	Källa
ABC 802	CAS: på `$736D–$79C0`	den disassemblerade ROM:en
ABC 800 M	ROM 0: CASRD `$0549`, CASWR `$05DF`, CASCTC `$0666`	jämförd med ABC 802
ABC 800 C	ROM 0, omkring `$2F` lägre	
ABC 806	ROM:en finns inte här (?)	manualerna

7 Innehållet

Ett program sparat med `SAVE` är BAC, kod som interpretatorn kör: med första byten `$82` från ABC80 och `$8F` från BASIC II, som har ett annat format. Med `LIST` och `PRINT #` blir det text, ra-

der som slutar med CR. LF skrivs inte, och två eller flera blanksteg i följd blir `09` n. Texten står i datablockens 253 byte, högst 252 tecken och sedan ETX.

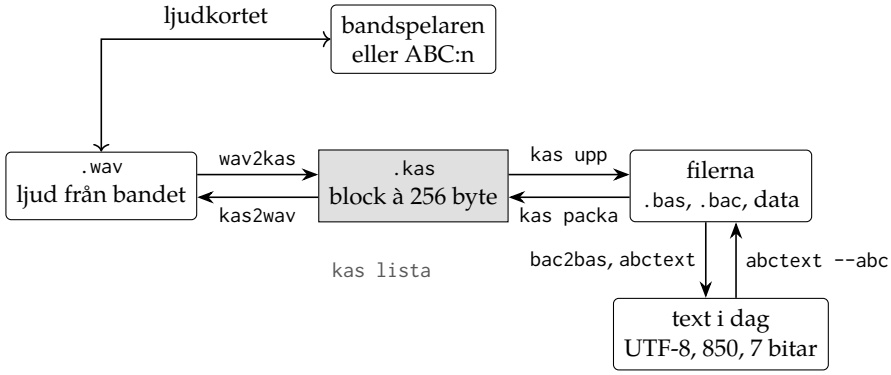


Figur 10. En rad i ett textblock. De fyra blankstegen blir `09 04`. ETX avslutar texten i blocket, och vid läsning är `00` slut på filen.

Bokstäverna ÄÖÅÜÉ är den svenska 7-bitarskoden: `[\]` blir ÄÖÅ, `{|}` äöå, `@` É, `` ` `` é, `^` Ü och `~` ü. I dag skrivs filerna oftast i UTF-8 med LF, och i MS-DOS i teckentabell 850 med CR LF.

8 Verktygen

Verktygen finns i Python 3 (`python/`) och i C89 (`c/`), med samma namn, flaggor och utdata. Mellanformatet är `.kas`: blockens 256 byte efter varandra, utan inramning.



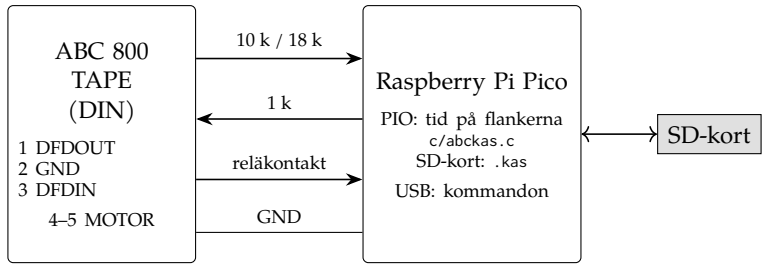
Figur 11. Verktygen och formaten. `wav2kas` läser både ABC80 och ABC 800-serien och väljer själv tidkonstanten ur filhuvudet. `kas2wav --abc800` skriver i ABC 800-seriens takt, och `--tk` väljer tidkonstanten för datablocken.

Några exempel:

Kommando	Gör
`wav2kas -o band.kas band.wav`	ljudet blir block
`kas lista band.kas`	filerna och blocken
`kas upp -d ut band.kas`	varje fil för sig
`bac2bas ut/PROG.BAC`	BAC blir text, som `LIST`
`abctext ut/PROG.BAS`	7 bitar och CR blir UTF-8 och LF
`kas packa -o ny.kas PROG.BAS`	en fil blir block, som ROM:en packar
`kas2wav --abc800 --tk 40 ny.kas`	block blir ljud för ABC 802 i 2 400 baud

9 Hårdvaran

Två förslag, inte byggda (se `HARDVARA.md`). Det första är en kabel till ljudkortet: en delare ned till linjenivå från DFDOUT, och ett motstånd in till DFDIN, där C44 i datorn tar bort likspänningen. ABC80 behöver troligen en komparator på ingången (?). Det andra är en Raspberry Pi Pico med ett SD-kort som tar bandspelarens plats och använder samma kod som verktygen.



Figur 12. Picon som bandspelare. Delaren tar DFDOUT från 5 V ned till 3,2 V. Motorkontakten drar en ingång till jord när datorn vill ha bandet: kommer det signal spelar Picon in, annars spelar den upp nästa fil.

Källor

ROM:erna för ABC80 (`abc80old.asm`: CASPRE, CASWBL, CASWBI, CASRBL, RCVSTA, CASINT, RCVBLK), ABC 802 (CASRD, CASWR, CASCTC) och ABC 800 M och C (ROM 0). Luxor, schemat för ABC 800, blad 7, "Processor unit" 55 10788. Luxor, *ABC 800 bruksanvisning*, avsnitten om ABC 820. Luxor, servicemanualen för ABC80. Gunnar Markesjö, *Mikrodatorns ABC*, 1978, s. 112–124, 164–167 och 193–194. Provet mot de elva riktiga banden och emulatorerna står i `README.md`.